```
package main

import (
      "fmt"
      "net/http"
      "encoding/json"
)

// http.ResponseWrite used to send req to client
// r contains the info about the incomming request
//r-->r.Method,r.URL,r.Header,r.Body
func Hello(w http.ResponseWriter,r *http.Request){

}




/*
for headers
w.WriteHeader(404)
fmt.Fprintln(w,"User not found")

or

http.Error(w,"User not found",http.StatusMethodNotAllowed)


*/
func users(w http.ResponseWriter,r *http.Request){
     id:=r.URL.Query().Get("id")

      if r.Method!=http.MethodGet{
            fmt.Fprintln(w,"Method not allowed")
        return
      }


      w.WriteHeader(http.StatusOK)
      fmt.Fprintln(w,"Ok",id)
}

func handleRoutes(w http.ResponseWriter,r *http.Request){
    if r.Method==http.MethodGet{
            fmt.Fprintln(w,"Get route")
      }
      if r.Method==http.MethodPost{
            fmt.Fprintln(w,"Post route")
      }
  http.Error(w,"Method not allowed",http.StatusMethodNotAllowed)
}


//Reading URl && Query parameter
/*
r.URL.Path  ->get the path


r.URL.Query().Get("id")--->get the query parameter id (?id=9)

id := r.URL.Query().Get("id")
name := r.URL.Query().Get("name")   ---> /users?id=123&name=aditya

*/
```

```
//Headers

/*
getting
userAgent := r.Header.Get("User-Agent")


setting

w.Header().Set("Content-Type","application/json")

func hello(w http.ResponseWriter, r *http.Request) {

      w.Header().Set("Content-Type", "application/json")

      fmt.Fprintln(w, `{"message":"hello"}`)

      clinet recives->Content-Type: application/json
}
*/
type User struct{
      ID int `json:"id"`
      NAME string `json:"name"`
}

func Home(w http.ResponseWriter,r *http.Request){

     w.Header().Set("Content-Type", "application/json")
       u:=User{
            ID: 3,
            NAME: "Vaibhav ",
       }
       json.NewEncoder(w).Encode(u)

}

//returning json-->encoding/json
/*
json.NewEncoder(w).Encode(u)
The encoder converts your struct into JSON and writes the resulting bytes to w.
Go's HTTP server sends that response to the client.

Also, Encode() itself returns an error, but you're choosing not to capture it in
this example.



*/

//Reading JSON request body

/*

var data MyStruct

err := json.NewDecoder(r.Body).Decode(&data)

if err != nil {
    // handle error
}


*/
```

```
type User2 struct {
      Name string `json:"name"`
}
func createUser(w http.ResponseWriter,r *http.Request){
      var user User2
      err:=json.NewDecoder(r.Body).Decode(&user)
      if err!=nil{
            http.Error(w, "Invalid JSON", http.StatusBadRequest)
            return
      }
      fmt.Fprintln(w,"User: ",user.Name)

}


//http.server
/*
server := &http.Server{
      Addr: ":8080",
}

server.ListenAndServe()


or


server := &http.Server{
      Addr:         ":8080",
      ReadTimeout:  5 * time.Second,
      WriteTimeout: 10 * time.Second,
      IdleTimeout:  30 * time.Second,
}

server.ListenAndServe()
*/

//ServeMux
/*

mux := http.NewServeMux()

mux.HandleFunc("/", home)
mux.HandleFunc("/users", users)

server := &http.Server{
      Addr:    ":8080",
      Handler: mux,
}

server.ListenAndServe()
             HTTP Request
                  |
                  ↓
             :8080 Server
                  |
                  ↓
                mux
              /     |     \
             /      |      \
            ↓       ↓       ↓
         home   users   products
*/
```

```
//15. Multiple handlers
/*
	mux.HandleFunc("/", home)
	mux.HandleFunc("/users", users)
	mux.HandleFunc("/products", products)

*/

//Middleware
//Middleware is a function that executes before or after your main HTTP handler
to perform common tasks.


func logging(next http.Handler) http.Handler{
	return http.HandlerFunc(func(w http.ResponseWriter,r *http.Request){
		fmt.Println("this is middleare",r.Header)
		next.ServeHTTP(w,r)
	})
}


//Goroutines

/*
One reason Go is excellent for servers is concurrency.
You don't manually create a goroutine for every HTTP request.
The HTTP server handles requests concurrently.

Client 1 ─────┐
Client 2 ─────┼──→ Go HTTP Server
Client 3 ─────┤
Client 4 ─────┘
                    |
              concurrent handlers
                    |
          ┌─────────┼─────────┐
          ↓         ↓         ↓
       handler1  handler2  handler3


*/
func main(){
  //creating your own route
	mux:=http.NewServeMux()

	mux.HandleFunc("/",Home)
	mux.HandleFunc("/user/",users)
	mux.HandleFunc("/createUser",createUser)

	mux.HandleFunc("/handleReq",handleRoutes)


	handler:=logging(mux)

	server:=&http.Server{
		Addr: ":8000",
		Handler:handler ,
	}
	server.ListenAndServe()

}
```